Trabajo Práctico III: Clean Code y Code Review

Alumno: Ignacio Figueroa
Tema: Refactoring seguro y revisión de código
Caso: Módulo de Facturación Médica, Hospital Central

1. Justificación de Mejoras y Refactoring Seguro (Partes A y B)

El código base presentaba un único método con baja cohesión, nombres crípticos, alto acoplamiento a tipos primitivos y números mágicos sin contexto de negocio. Se aplicaron tres técnicas formales de refactoring seguro:

Introduce Parameter Object:

Problema que resuelve: La firma original recibía múltiples argumentos sueltos (List<Double> estudios, boolean p, boolean o, int t), lo que dificultaba entender qué representaba cada parámetro e incrementaba el riesgo de errores por orden.

Mejora: Se creó la clase SolicitudFacturacion, encapsulando los datos del paciente y la consulta en un objeto coherente y tipado.

Extract Method:

Problema que resuelve: El método original acumulaba en un solo bloque la suma de aranceles, el cálculo de prepaga, la bonificación por obra social y el descuento por frecuencia de turnos.

Mejora: Se dividió el flujo en métodos privados autodescriptivos (calcularSubtotalEstudios, aplicarDescuentoObraSocial, aplicarDescuentoTurnosFrecuentes), facilitando la lectura paso a paso y la reutilización.

Move Method:

Problema que resuelve: La clase central manejaba directamente los porcentajes y valores fijos sin delegar el tratamiento de datos propios del paciente o los estudios.
Mejora: Se reubicaron las validaciones de entrada y el acceso a los datos dentro de la nueva estructura del dominio, centralizando constantes como COBERTURA_PREPAGA_PORCENTAJE y UMBRAL_TURNOS_FRECUENTES

2. Checklist de Code Review (Parte C)

Pregunta de Control	Estado Inicial	Observaciones Técnicas
¿Los nombres son descriptivos?	❌ No	Parámetros y variables como `p`, `o`, `t`, `r`, `e` y `calc` no comunican su significado dentro del dominio de negocio.
¿Las funciones poseen una única responsabilidad?	❌ No	Un solo método acumula el cálculo de estudios, la liquidación de cobertura médica, la bonificación por convenio y el descuento por turnos frecuentes.
¿Existe duplicación innecesaria?	⚠️ Parcial	La iteración contiene sumas condicionales redundantes en lugar de modularizar el costo individual.
¿El código es fácilmente extensible?	❌ No	Agregar una nueva regla o promoción obliga a reescribir la secuencia entera de condicionales.

¿Hay manejo correcto de errores?	❌ No	Si la lista `estudios` es nula lanza `NullPointerException`, y el resultado final puede quedar en valores negativos.
¿Existe acoplamiento innecesario?	❌ No	Alto acoplamiento a valores primitivos y booleanos dispersos en la firma del método.
¿La cohesión es adecuada?	❌ No	Baja cohesión interna: la clase concentra tareas dispares sin delegar responsabilidades.
¿Se respetan principios SOLID?	❌ No	Viola el Principio de Responsabilidad Única (SRP) y el Principio de Abierto/Cerrado (OCP).
¿El código resulta legible para otro desarrollador?	❌ No	Presenta números mágicos (`0.5`, `0.9`, `100`, `3`) y comprobaciones redundantes como `if (p == true)`.

3. Feedback Profesional del Code Review (Parte D)

Revisión de Código Pull Request: Módulo de Facturación

- Nomenclatura y Expresividad: Se identificaron identificadores de un solo caracter (p, o, t, r) que dificultan la comprensión del negocio. Se recomienda utilizar nombres claros como poseePrepaga, poseeObraSocial y cantidadTurnosPrevios.
- Constantes Nombradas: Los literales 0.5, 0.9, 100 y 3 operan como números mágicos. Conviene extraerlos a constantes de clase para documentar el porcentaje o regla que representan y facilitar futuras actualizaciones tarifarias.
- Modularidad y Cohesión: La función calc reúne tres reglas de negocio independientes. Se sugiere aplicar Extract Method para aislar cada cálculo en funciones privadas y utilizar un Parameter Object para evitar el traspaso de múltiples primitivos.
- Validación de Entradas: Conviene incorporar validaciones defensivas frente a colecciones nulas o vacías para evitar excepciones en tiempo de ejecución.